

Eight takeaways, one per enabling objective

1 When a prompt fails, the failure type tells you which technique is missing.

Output in the wrong shape points to a missing output constraint, drift across turns points to an underspecified system prompt, and a hallucinated structure points to the absence of few-shot examples. The instinct to reword the instruction and try again rarely works, because none of those failures are phrasing problems. Diagnose the failure type first, then add the technique that addresses it. When prompt-level instructions are not enough because untested inputs still break the parser, move output control into the API with structured outputs: JSON outputs constrain the final response against a schema, and strict tool use validates the arguments Claude passes to your tools, at the cost of first-call compilation latency and added input tokens.

2 Match the reasoning depth to the task before you tune the prompt.

Enable reasoning only where a reasoning pass changes the answer and calibrate the effort setting to the problem rather than raising it on every call. Remember that thinking blocks return to the API unchanged or the next request fails. Choosing which model to run, as distinct from whether to enable reasoning, is taught in the MSO Foundations module that precedes this one.

3 A stream ending is not a message completing.

Streaming buys perceived latency at the cost of assembling the response yourself from partial events. Act on a block only after it closes, commit a turn to history only after `message_stop`, and on an interrupted stream discard the partial turn and retry. The failure mode to recognize is a tool-use error on a retry that traces back to a half-built block from a dropped stream, not to the schema.

4 Every wrong-tool selection traces back to the schema, and most of the time to the description.

Claude picks a tool by reading the description field and matching it against the user's request, which means two tools that both say "use this to find information" are indistinguishable from Claude's side even when the input schemas look nothing alike. The one sentence that resolves most wrong-tool bugs is the exclusion condition: a line in every description naming when not to call the tool, written into the schema at design time rather than after the first wrong call shows up in a log. When someone else has already written the tools, MCP lets you connect a maintained server instead of authoring every schema by hand, but each connected server adds its tool definitions to the context window whether the tools are used, so connect deliberately and control loading cost.

5 Context is a fixed budget, and tool outputs spend it faster than anything else in the loop.

Production tool outputs run three to five times longer than the fixtures used in development, so a session that holds together cleanly across fifty turns in testing can hit the ceiling at turn eight once it ships. Pruning, compaction, and subagent handoffs each buy back headroom in different ways, and the one to apply depends on whether you still need the earlier state. When tool selection starts degrading after a fixed number of turns, the window is the first place to look, not the schema.

6 The workflow-or-agent decision sets the cost of everything that follows, and human checkpoints belong in the design.

A workflow is the right call when you can write the exact steps in code, and an agent is the right call when you can specify the goal and the tools but not the path between them. Choosing wrong in either direction only surfaces in production: agents where workflows would do add context cost and behavior that lives in transcripts, and workflows where agents are needed break the first time an input falls outside the path. If a tool can take an irreversible action, the human-in-the-loop checkpoint goes in before the loop is wired, not after the first write reaches a customer environment.

7 Memory scope is decided by the shape of the session, not by what is easiest to implement.

In-context memory is the simplest pattern to write, which is why it is also the one that fails earliest when production sessions turn out to be shorter and more numerous than the long continuous sessions used in development. External storage adds latency but the state survives across sessions, summarized memory cuts cost but loses anything the summarizer prompt did not preserve, and stateless is correct for jobs that complete and close. The refactor from in-context to external under production pressure takes about an hour, and making the same choice deliberately at design time takes about twenty minutes. Carrying repeatable instructions across tasks is a separate problem from carrying state, and the pattern for it is a Skill: a markdown file Claude loads on demand by matching its description, rather than instructions injected into every session.

8 Calculate the cost of a multimodal input before you write the ingestion code and match the API to the workload.

An image costs $[\text{width} / 28] \times [\text{height} / 28]$ visual tokens, and the per-image ceiling differs by model tier. A high-resolution original on the newest models can cost many times what a thumbnail costs in your test set, so the formula needs to run against the largest input you expect in production rather than the inputs you have on hand. Inline base64 fits one-off images, the Files API fits assets reused across requests, and the Message Batches API handles offline work at lower per-token cost in exchange for non-deterministic latency. The mistake worth avoiding is calling the synchronous API in a loop and treating that as batching.

WHAT COMES NEXT

This module established the Developer primitive library, including five interaction types that all subsequent Developer modules draw from. The patterns introduced here, which include prompting craft, tool schemas, context engineering, agent construction, memory scoping, and multimodal ingestion, form the foundation for every module that follows.